This program is a **basic Retrieval-Augmented Generation (RAG) pipeline** built with LangChain. Its job is to load a local document, split it into smaller pieces, convert those pieces into embeddings, store them in a vector index, retrieve the most relevant chunks for a question, and then ask an LLM to answer using only that retrieved context.

What the program is doing overall

Think of the flow as:

File → chunks → embeddings → vector search → retrieved context → LLM answer

That is the standard RAG pattern. The point is to make the model answer from your document, not from its general memory.

1) Package installation

Your install command is meant to bring in all dependencies for:

- loading files
- splitting text
- embedding text
- storing vectors in FAISS
- calling the OpenAI chat model

Important issue

langchain.chains is **not** a package you install with pip. It is an import path used in some versions of LangChain, not a separate library. So that part of the install line is likely wrong.

What else matters

This program also assumes:

- you have an OpenAI API key set
- the file Sample.txt exists in the same folder/notebook path
- the environment can call OpenAI for embeddings and chat completion

2) Imports

Each import has a specific role:

- os
  Used to check whether the file exists.

- TextLoader, PyPDFLoader
  Used to read documents into LangChain Document objects.
  In your current code, only TextLoader is actually used.
- RecursiveCharacterTextSplitter
  Breaks long text into manageable chunks.
- OpenAIEmbeddings
  Converts each chunk into a numeric vector.
- ChatOpenAI
  Sends the final prompt to the LLM for answer generation.
- FAISS
  Stores embeddings in a searchable vector database in memory.

Small code quality note

PyPDFLoader is imported but not used. That is harmless, but unnecessary unless you plan to load PDFs too.

3) load_documents()

This function handles **data ingestion**.

What it does

- Creates an empty list called docs
- Checks whether Sample.txt exists
- If it exists, loads it using TextLoader
- Adds the loaded document(s) into docs
- If nothing is loaded, raises an error

Why this matters

RAG starts with source data. If no document is loaded, there is nothing to retrieve from, so the whole pipeline fails early.

Output

It returns a list of LangChain Document objects.
Each Document contains:

- page_content: the actual text

- metadata: extra information like file path

Practical implication

Your error message says:
“Add sample.pdf or sample.txt to the folder.”

But your code checks only Sample.txt. That mismatch can confuse users. Also note that file names can be case-sensitive depending on the OS.

4) split_documents(documents)

This function handles **splitting**.

What it does

It creates a RecursiveCharacterTextSplitter with:

- chunk_size=500
- chunk_overlap=100

Then it splits the loaded documents into smaller chunks.

Why chunking is needed

LLMs and embedding models work better when long text is broken into smaller pieces:

- retrieval becomes more precise
- embeddings represent narrower ideas more cleanly
- the final prompt stays smaller and more relevant

Why overlap is used

If one idea starts near the end of one chunk and continues into the next, overlap helps preserve meaning. Without overlap, important context may get cut off.

Trade-off

- Smaller chunks = better precision, less context
- Larger chunks = more context, more noise
- Overlap improves continuity, but increases storage and retrieval redundancy

Output

A list of smaller Document chunks.

5) create_vectorstore(chunks)

This function handles **embedding and indexing**.

What it does

- Creates an embedding model: text-embedding-3-small
- Converts each chunk into a vector
- Stores all vectors in a FAISS index

Why embeddings matter

Embeddings turn text into numbers in a way that captures semantic meaning.
This allows the system to find chunks that are conceptually similar to the user's question, even if wording is different.

Example:
A question about "steps of RAG" can still match a chunk that says "RAG workflow includes five stages."

Why FAISS is used

FAISS is a fast vector similarity search library. It lets you efficiently retrieve the nearest chunks for a given question.

Output

A vector store object that can later be turned into a retriever.

6) ask_rag(question, vectorstore)

This is the core **retrieval + generation** step.

Step A: Create retriever

vectorstore.as_retriever(search_kwargs={"k": 3})

This means:

- convert the vector store into a retriever
- fetch the top 3 most relevant chunks for the question

Why k=3

It is a simple balance:

- 1 chunk may miss context
- too many chunks may add noise
- 3 is a common starting point for small demos

Step B: Retrieve documents

docs = retriever.invoke(question)

This takes the user question, embeds it, compares it against stored chunk vectors, and returns the closest matching chunks.

Step C: Build context

The code joins the retrieved chunk texts into one string:
context = "\n\n".join(doc.page_content for doc in docs)

That combined text becomes the evidence sent to the model.

Step D: Build prompt

The prompt says:

- answer only from the context
- if the answer is not there, say you do not know

This instruction is important because it reduces hallucination.

Step E: Call the LLM

ChatOpenAI(model="gpt-4o-mini", temperature=0)

- gpt-4o-mini generates the answer
- temperature=0 makes output more deterministic and less creative

Step F: Return answer and sources

The function returns:

- response.content → the generated answer
- docs → the retrieved source chunks

That is useful because you can inspect what the model actually used.

7) main()

This function orchestrates the full workflow.

Execution order

1. Load document(s)
2. Split into chunks
3. Create vector store
4. Set a question
5. Retrieve relevant chunks
6. Generate answer
7. Print question, answer, and source metadata

Why this structure is good

It keeps the pipeline readable and modular:

- ingestion is separate
- splitting is separate
- indexing is separate
- QA is separate

That makes it easier to debug and upgrade.

8) What happens with your current question

Your active question is:

“How far the sun is from earth?”

Why this is a good RAG test

It tests whether the system is actually grounded in the document.

If your Sample.txt is about AI, machine learning, and RAG, then this answer is **not in the document**. So ideally:

- retrieval will still return the “closest” chunks
- but those chunks will not actually answer the question
- the prompt should push the model to say: **“I do not know”**

Why that behavior is correct

A good RAG system should not answer from unrelated world knowledge when instructed to use only the provided context. That is one of the main reasons people use RAG in the first place.

9) What gets printed in SOURCES

This loop prints metadata for each retrieved chunk:

- source file name/path
- maybe page number if present
- any other metadata LangChain stored

Why this matters

It helps with:

- traceability
- debugging retrieval quality
- explaining where the answer came from

In production, this becomes important for user trust.

10) Strengths of this program

This is a solid beginner RAG demo because it shows the full pipeline clearly:

- document loading
- chunking
- embedding
- indexing
- retrieval
- answer generation

It is simple enough to understand, but realistic enough to demonstrate the core RAG idea.

11) Limitations / meaningful flaws to fix

A few important ones:

1. Wrong pip package

langchain.chains should not be in the install command.

2. Unused import

PyPDFLoader is imported but not used.

3. File-name mismatch

Code uses Sample.txt, but the error message mentions sample.txt and sample.pdf.

4. No API key handling

The code assumes credentials are already configured.

5. Prompt-only grounding

The model is told to use only context, but prompt instructions alone are not perfect. Sometimes models still infer more than you want.

6. No citation formatting

You print metadata after the answer, but the answer itself does not cite which chunk supported it.

7. Small demo scale

FAISS in-memory is fine for demos, but not enough for larger production use cases.

12) Best way to explain this program in one sentence

This program builds a small searchable knowledge base from a local text file and uses an LLM to answer questions based only on the most relevant retrieved text chunks.

13) Recommended next improvements

If you want to make this stronger, I'd do these in order:

1. **Fix the install command**
2. **Align file names and error messages**
3. **Print retrieved chunk text for debugging**
4. **Add source citations into the answer**
5. **Support both TXT and PDF ingestion**
6. **Use a prompt template instead of a raw f-string**
7. **Persist the FAISS index for reuse**